2026 嵌入式芯片与系统设计竞赛 · 平台三

安路 PH1P100 × 瑞芯微 RK3588

异构图像加速平台

MIPI 采集 · 四路硬件预处理 · 硬件 JPEG · PCIe 传帧 · NPU 推理

主芯片：　安路 PH1P100SBG484（FPGA）+ 瑞芯微 RK3588
开发工具：　Anlogic TD Release_2026.2_NL / RKNN-Toolkit2 2.3.2
互联方式：　PCIe Gen2 ×2，BAR0 1 MiB
文档版本：　2026-10-30（对应位流 `9c24eb71`）

目 录

1 作品概述 5
1.1 系统架构 5
2 赛题要求对照 6
2.1 测试平台与配置 6
2.2 帧契约 6
3 MIPI 采集与运行时曝光 7
3.1 4K30 全帧采集 7
3.2 运行时曝光控制 7
4 硬件预处理流水线 8
5 硬件 JPEG 压缩 9
5.1 方案选择 9
5.2 码流契约 9
5.3 实测结果 9
6 PCIe 传输与延迟 11
6.1 传输带宽 11
6.2 端到端延迟 11
7 RKNN 推理与叠加 12
8 资源、时序与稳定性 13
9 结论与已知边界 14
9.1 复现方式 14

图目录

图 1 硬件 JPEG 压缩流经 PCIe 回传、主机解码后送入 RKNN 的检测叠加结果（2026-10-30 板测：bottle 0.61、tv 0.52、tv 0.33） 5
图 2 Sobel 边缘检测的硬件输出：显示器轮廓、线缆、天花板桁架等结构清晰可辨 8
图 3 板端压缩、经 PCIe 回传后由主机封包解码的实拍图（1920×1088，熵流 93,443 B） 9

表目录

表 1 全链路各环节的硬件归属 5
表 2 赛题六项硬性交付的完成情况 6
表 3 测试平台配置 6
表 4 两种位流的帧契约；早期 1080p ROI 位流（slot 2,592,768 B）的数据不与本表混用 6
表 5 4K30 单路采集实测（2026-10-29） 7
表 6 同一场景、同一曝光下的增益扫描（2026-10-30 22:19，室内开灯）：均值随码值单调上升，说明写入确实落到传感器；绝对均值随场景亮度变化，不能跨场景比较 7
表 7 四路算法在同一场景、同一曝光下的统计对比（2026-10-30 22:35，拉普拉斯均值表征高频能量）。绝对均值随场景亮度变化，四路之间的相对关系才是判据 8
表 8 硬件 JPEG 压缩的实测指标 9
表 9 码率—画质曲线（主机参考编码器，与 Pillow 交叉验证偏差 ⩽ 1.5%） 10
表 10 端到端延迟分布（2026-10-30 21:24 实测） 11
表 11 NPU 推理实测 12
表 12 两个位流的资源占用与时序；两者功能不同，数字不可混用 13
表 13 距完整交付的剩余缺口与对应动作 14

1 作品概述

本作品在国产 FPGA 安路 PH1P100 与瑞芯微 RK3588 之间建立一条完整的机器视觉前端链路：IMX415 传感器经 MIPI CSI-2 把 4K 原始数据送入 FPGA，FPGA 内部完成帧校验、四路并行预处理与硬件 JPEG 压缩，压缩码流经 PCIe 送回 RK3588，由 NPU 完成目标检测并把结果叠加回画面。 采集、预处理、压缩、推帧四个环节全部在 FPGA 内部用 RTL 实现，RK3588 只承担取流、显示、 推理与人机交互，符合赛题对“核心链路须在 FPGA 内自主实现”的约束。



图 1　硬件 JPEG 压缩流经 PCIe 回传、主机解码后送入 RKNN 的检测叠加结果（2026-10-30 板测：bottle 0.61、tv 0.52、tv 0.33）

1.1 系统架构

环节	实现位置与要点
MIPI CSI-2 接收	`rtl/ep_side/cam/` 与 `rtl/checked/`：4-lane 软 PHY + ECC/CRC/行数/包长裁决，整帧校验通过才放行
四路预处理	`rtl/ep_side/img/algo/`：灰度、3×3 高斯、直方图均衡、Sobel 边缘，共享一条 4 像素/拍输入流并行展开
硬件 JPEG	`rtl/ep_side/codec/`：DCT → 量化/之字形 → 规范哈夫曼，输出纯熵编码码流
PCIe 端点与推帧	`pcie_ep_engine.v` + `ddr_frame_pusher.v`：128 B 载荷 Posted MWr，one-shot/HELD/ACK 握手
收帧与显示	Linux 内核模块提供字符节点；Rust 侧做解包、后处理与 MJPEG 推流
NPU 推理	RKNN YOLO11n INT8，双输出量化，结果按归一化坐标叠加

表 1　全链路各环节的硬件归属

2 赛题要求对照

序号	赛题要求	状态	关键数据
1	MIPI CSI IP，至少单路 1080P@60fps，鼓励 4K@30fps	4K30 达成	3840×2160 @ 29.9 fps，丢组 0，像素吞吐 248 Mpix/s ≈ 2 倍 1080p60
2	不少于 3 种预处理，AXI-Stream 流水线	达成（类 AXI-Stream）	灰度 / 高斯 / HE / Sobel 四路并行，1920×1088 @ 30 fps
3	JPEG 或 LZ77 硬件压缩，压缩率 ⩾ 30%	达成，非实时	典型 95.5%, 编码 65.7 ms/帧, 7.5 8.5 fps（亮场）
4	PH1P100–RK3588 高速通道，端到端 ⩽ 50 ms	传输段达成, 端到端未闭合	压缩帧传输 < 1 ms；RAW 帧端到端 p05 33.2 ms、中位 73.2 ms
5	RK3588 NPU 本地推理并叠加	达成	YOLO11n INT8，40 45 ms/帧，全链路 422 486 ms/帧
6	完整量化测试报告	五项有数, 功耗未测	帧率、延迟、资源、压缩率、推理耗时均已实测；整机功耗缺测量手段

表 2　赛题六项硬性交付的完成情况

2.1 测试平台与配置

项	配置
FPGA	安路 PH1P100SBG484，TD `Release_2026.2_NL`
主机	Orange Pi 5 Max / RK3588，内核 7.2.0，NFS root
互联	PCIe Gen2 ×2（5 GT/s ×2，`LnkSta` 实测），BAR0 1 MiB
传感器	2 × IMX415（FEP-VID006-MIPI 子卡），27 MHz / 891 Mbps × 4 lane
时钟	clk_ddr 100 MHz（实测 99.92 MHz）、MIPI byte clk 187.5 MHz、tl_clk_user 62.5 MHz
位流	4K 采集位流与 1920×1088 ROI/JPEG 位流，二者独立，均为 SRAM 配置

表 3　测试平台配置

2.2 帧契约

量	4K 采集位流	1920×1088 位流（ROI/JPEG）
图像格式	3840×2160 packed RAW10	1920×1088 packed RAW10（算法侧解包为 8 bit 灰度）
行宽 stride	4,800 B	2,400 B
payload	10,368,000 B	2,611,200 B（解包后 8 bit 为 2,088,960 B）
整帧长度	10,371,072 B（2,532 × 4 KiB）	2,613,248 B（638 × 4 KiB）
每帧 MWr	81,024 笔 128 B + 门铃	20,416 笔 128 B + 门铃
DDR 双缓冲	A0/A1 = 0x0100_0000 / 0x0200_0000	同左，B 路 0x0300_0000 / 0x0400_0000

表 4　两种位流的帧契约；早期 1080p ROI 位流（slot 2,592,768 B）的数据不与本表混用

3 MIPI 采集与运行时曝光

3.1 4K30 全帧采集

测量项	A 路结果
分辨率	3840×2160 packed RAW10，stride 4,800 B
几何	全阵列 3,864×2,192; 有效图 3,840×2,160; `X_OFF=12`、`SKIP_ROWS=16`，物理原点 (12, 16)
每帧组数 `0x1A0`	2,073,600（= 2160 × 960，与理论值一致）
丢组 `0x1A4`	**0**
AXI 拍数 `0xF0`	324,096（= 10,371,072 B / 32 B）
帧率	**29.9 fps**
R10F 校验	payload 与 header 双层逐字节复算通过

表 5　4K30 单路采集实测（2026-10-29）

需要说明的是：本平台没有实现 1080p60 的传感器时序。4K30 的像素吞吐（248 Mpix/s）是 1080p60（124 Mpix/s）的两倍，但帧时序不同，因此报告中不把 4K30 写成“已满足 1080p60”。

3.2 运行时曝光控制

传感器模拟增益做成 BAR0 寄存器, 主机可在不重下位流的前提下调整曝光(白天室内建议 0x20–0x30, 夜间 0x50–0x64)：向 `0x280` 写入增益码 （IMX415 每 LSB 约 0.3 dB)，硬件锁存后经 FPGA 内部的 AE 状态机写 I2C 寄存器 `0x3090`； `0x284` 回读 `\{done_b, done_a, ae_st, applied_gain\}` 供主机确认写入完成。

增益码	等效增益	画面均值	最大值
`0x00`	0 dB	21.6	255
`0x20`	9.6 dB	38.5	255
`0x40`	19.2 dB	76.7	255
`0x64`	30.0 dB	166.1	255（多处饱和）

表 6　同一场景、同一曝光下的增益扫描（2026-10-30 22:19，室内开灯）：均值随码值单调上升，说明写入确实落到传感器；绝对均值随场景亮度变化，不能跨场景比较

4 硬件预处理流水线

四路算法在同一个 `img_algo_parallel_top` 内并行展开，共享一条 4 像素/拍的 RAW10 输入流； 未选中的三路照常产生输出后被丢弃（不反压），只有被选中的那一路把反压传回 ROI 适配器。 主机通过 `0x240[2:1]` 选择输出：`0x0` 灰度、`0x2` 高斯、`0x4` 直方图均衡、`0x6` Sobel。

0x240	算法	均值	标准差	拉普拉斯均值
0x0	灰度	54.3	48.1	3.07
0x2	3×3 高斯	54.4	48.0	**1.03**
0x4	直方图均衡	**131.6**	72.6	9.17
0x6	Sobel 边缘幅值	10.3	18.5	5.08

表 7　四路算法在同一场景、同一曝光下的统计对比（2026-10-30 22:35，拉普拉斯均值表征高频能量）。绝对均值随场景亮度变化，四路之间的相对关系才是判据

高斯滤波把高频能量从 3.07 压到 1.03（降低 66%），直方图均衡把均值抬到 131.6 并显著增强 局部对比，Sobel 只保留边缘、画面均值降到 10.3，三者相对灰度基准的变化方向都符合算法预期。



图 2　Sobel 边缘检测的硬件输出：显示器轮廓、线缆、天花板桁架等结构清晰可辨

Sobel 的定点实现为 $G = \min\bigl(255, (|G_x| + |G_y|) \gg 1\bigr)$，其中两级梯度都用 1-2-1 的行和与列和之差得到， 全程没有乘法和除法，与输入同速（1 拍 4 像素），因此四路算法可以完全不增加流水线节拍。

5 硬件 JPEG 压缩

5.1 方案选择

在白天的 1920×1088 灰度帧（2,088,960 B）上，几种无损方案都没有达到 30% 的压缩率：差分加零游程 RLE 反而膨胀 8.45%，差分加 LZ77 只有 3.54%，3×3 高斯预处理后再 LZ77 也只有 17.26%。DCT 量化直接丢弃人眼不敏感的高频，压缩率由量化表控制，因此最终选择灰度基线 JPEG。

5.2 码流契约

FPGA 只输出熵编码字节，不含 SOI/APP0/DQT/SOF0/DHT/SOS/EOI：这些头部由主机用固定模板补齐，封装成标准 JFIF。内部流程为 8×8 分块、像素减 128、Q12 定点二维 DCT、标准亮度量化表、之字形扫描、DC 差分与 AC 游程、T.81 附录 K 规范哈夫曼编码，并在每个 `0xFF` 字节后补 `0x00`。

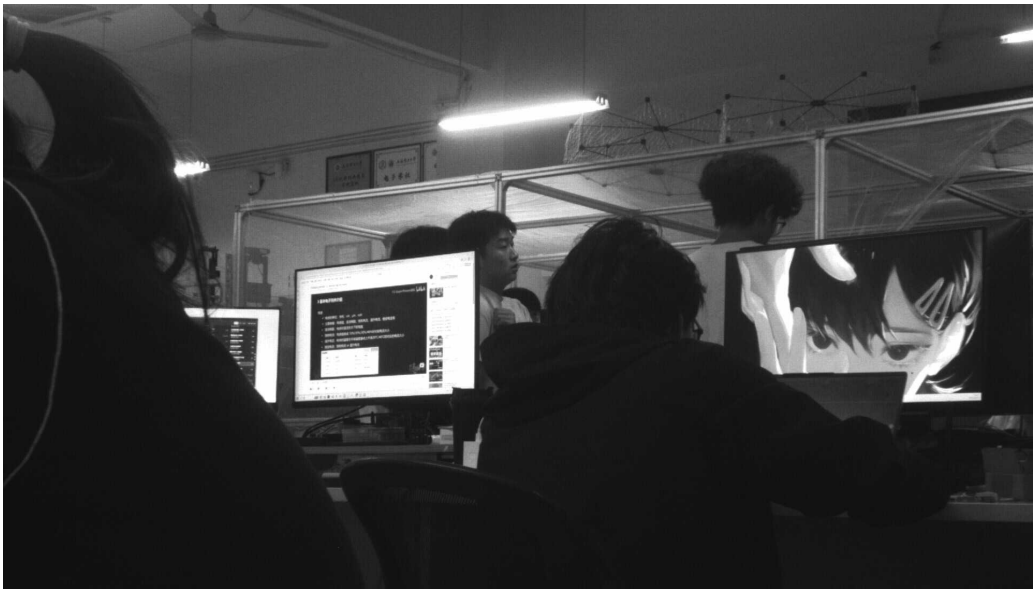


图 3　板端压缩、经 PCIe 回传后由主机封包解码的实拍图（1920×1088，熵流 93,443 B）

5.3 实测结果

项目	结果
典型熵流与压缩率	93,443 B / **95.53%**（22.4 ×）
长稳码流范围	35.8 – 110.8 KB，压缩率 94.7% – 98.3%
编码器吞吐	**65.7 ms/帧**（6.57 M 周期 @ 100 MHz），首字节延迟 0.36 ms
整帧吞吐（亮场）	7.5 – 8.5 fps；1 h 长稳 26,965 帧 = 7.49 fps，`jb_done` 严格单调
整帧吞吐（暗场）	4.07 fps；2026-10-30 复测 6.45 fps（45 s / 291 帧，码流 69.2 – 69.5 KB）
整帧 PSNR	34.9 dB（同场景参考帧），静态区域 34 – 40 dB

表 8　硬件 JPEG 压缩的实测指标

质量因子 q	熵流字节	压缩率	PSNR
20	55,138	97.36%	34.5 dB
30	70,874	96.61%	35.6 dB
50	105,130	94.97%	37.1 dB
70	163,348	92.18%	39.2 dB
90	349,666	83.26%	44.5 dB

表 9　码率—画质曲线（主机参考编码器，与 Pillow 交叉验证偏差 ⩽ 1.5%）

6 PCIe 传输与延迟

6.1 传输带宽

4K RAW 整帧 10,371,072 B 需要 81,024 笔 128 B 的 Posted MWr，打包、DDR 搬运与 PCIe 传输 合计约 20 ms，折合 **519 MB/s**，接近 Gen2 ×2 的可用带宽。压缩帧只有约 93 KB， 传输耗时 **小于 1 ms**，主机取流约 2.4 ms，链路本身不是瓶颈。

6.2 端到端延迟

主机连续执行 `link_ctl arm/ack` 与 `frame_grab` 取 23 帧有效样本，用 R10F 头部记录的 SOF 时间戳与本机 `0×238/0×23C` 自由计数器做差，得到从传感器帧首到主机拿到整帧的延迟：

统计量	1920×1088 RAW 帧
p05 / 最小	**33.2 ms**
中位	73.2 ms
p95 / 最大	113.6 ms

表 10　端到端延迟分布（2026-10-30 21:24 实测）

延迟不是单一数字，原因在于当前推帧是 one-shot：每次 ARM 只推一帧，主机 ARM 的相位 相对 30 fps 帧同步是随机的，最坏要等一整帧（33.3 ms）才有新帧发布。因此延迟在 “一帧周期 – 两帧周期加传输”之间分布，而传输段本身小于 1 ms。

据此，赛题要求的“端到端 ⩽ 50 ms”应当分两个口径表述：传输段满足，端到端不满足。要真正压进 50 ms，需要同时做两件事：把 one-shot 改成流式推帧（去掉等帧的 33 ms）， 并把编码器从 65.7 ms 降到约 20 ms（当前 DCT 的 8 行与 8 列串行、熵编码每系数 3 拍是瓶颈）。

7 RKNN 推理与叠加

项目	结果
模型	YOLO11n 640 INT8，双输出（`boxes [1,4,8400]` + `scores [1,80,8400]`）
推理耗时	**40 – 45 ms/帧**（含反量化）；离线单帧 50.8 ms
双路 960×544	21.9 pair/s，A 路 6 框、B 路 7 框，推理错误 0
压缩流全链路	解码 6 – 28 ms + 推理 44 ms，端到端 422 – 486 ms/帧
叠加方式	按归一化坐标绘制，改变推流分辨率不影响框位置

表 11　NPU 推理实测

Ultralytics 默认导出的单个 `[1,84,8400]` 输出无法直接量化为 INT8：框坐标的数值范围主导了 整张量的量化尺度，导致 0 1 的类别分数全部被量化为 0。本作品在 ONNX 最后一层把 boxes 与 scores 拆成两个输出，RKNN 分别计算量化参数后，类别分数恢复正常（最大值 0.87）。

8 资源、时序与稳定性

位流	LUT6	FF	ERAM	DSP	时序
JPEG 位流（含 Sobel）	29,822（41.6%）	29,441（20.5%）	160（53.3%）	38（17.3%）	Setup 0.004 ns，Hold 0.020 ns
4K 采集位流	17,548（24.5%）	—	136	0	SWNS 0.268 ns

表 12　两个位流的资源占用与时序；两者功能不同，数字不可混用

时序余量只有 4 – 101 ps，属于擦边通过，主要压力点在 187.5 MHz 的 MIPI 字节时钟。任何 RTL 或约束改动都必须重新查看 `final_timing.rpt`，不能把某一次的余量当作稳定余量。

长期稳定性方面，压缩支路完成过 1 小时连续压缩（26,965 帧、7.49 fps，`jb_done` 严格单调，无死锁）；双路 1080p 自由采集 60.6 s 内 A/B 各 1,819 帧、丢组 0；双路 pair 取帧连续 600 次全部成功，两路 `capture_seq` 完全一致。

9 结论与已知边界

六项硬性交付中，预处理、压缩、NPU 推理三项证据完整；采集以 4K30 覆盖了 1080p60 的像素吞吐但缺少 60 fps 帧时序；高速通道的传输段满足 ⩽ 50 ms，端到端尚未闭合；测试报告缺少整机功耗。

优先级	缺口	下一步
P0	整机功耗完全空白，板上无电流传感器	外接 USB-C PD 功率计或台式电源，测待机 / 4K / JPEG / NPU 四种状态
P0	端到端未进 50 ms	流式推帧去掉等帧时间，同时把编码器降到约 20 ms
P1	没有 1080p60 时序	新增 1080p60 寄存器表并在板验证
P1	4K 与 JPEG 是两个独立位流	做 4K ROI 压缩，或让编码器吞吐超过 30 fps
P2	双路 4K、4K 硬件同步、MSI、故障注入	先解决打包节拍与板级 XVS/XHS 走线

表 13　距完整交付的剩余缺口与对应动作

9.1 复现方式

```
# 1) 下载 SRAM 位流（本机 TD），随后重启 RK3588 才会重新枚举 Gen2 x2
cd fpga && /opt/anlogic/TD_Release_2026.2_NL/bin/bw.sh download_sram.tcl
ssh root@<BOARD> 'sync; reboot'

# 2) 按位流几何加载驱动
ssh root@<BOARD> 'insmod /root/anlogic_pci.ko frame_width=1920 frame_height=1088
frame_slots=1'

# 3) 一键自检
tools/board/demo.sh env      # 链路、驱动、温度、设备节点
tools/board/demo.sh cap      # 取一帧
tools/board/demo.sh npu      # 压缩 → PCIe → 解码 → RKNN → 画框

# 4) 运行时曝光与算法切换
/root/pci_bar 0 0×280 0×30  # 传感器增益（0.3 dB/LSB）
/root/pci_bar 0 0×240 0×6   # 0×0 灰度 / 0x2 高斯 / 0x4 HE / 0×6 Sobel
```